Gestión de eventos e inscripciones

Informe técnico del proyecto integrador

Base de Datos Aplicada · MongoDB, FastAPI, Docker y análisis con Python

Fecha de cierre técnico: 29 de octubre de 2026

Resumen

Implementamos una solución documental para administrar eventos con sede y actividades heterogéneas, registrar inscripciones y controlar cupos. El modelo combina datos embebidos y referencias; la API aplica validaciones y transacciones para mantener consistencia entre inscripciones y vacantes.

La carga reproducible contiene 20 eventos y 200 inscripciones. Tras la demo se conservaron 22 eventos y 201 inscripciones, sin errores en nueve controles de integridad. Se ejecutaron doce consultas, se compararon tres índices y se desarrollaron tres indicadores con dos gráficos. El notebook se ejecutó con seis celdas y cero errores guardados.

Los resultados describen un poblado ficticio. La ejecución secuencial y las comprobaciones presentadas no sustituyen pruebas de carga concurrente ni recuperación ante fallos.

Contenido

0 Contexto y alcance

1 Diseño documental y diccionario

2 Consultas MongoDB

3 API y consistencia

4 Índices y planes de ejecución

5 Indicadores y visualizaciones

6 Reproducción de la solución

7 Conclusiones y anexo de evidencias

0 Contexto y alcance

Problema

Una organizacion necesita publicar eventos con sede, cupo, precio y cronograma, registrar inscripciones y cancelaciones, y consultar demanda, ocupacion y recaudacion. Los registros deben conservar su historial y evitar duplicados o cupos inconsistentes.

Actores

- Organizador: crea borradores, publica, modifica o cancela eventos y consulta inscripciones.
- Usuario: se identifica mediante DNI y aporta nombre, email y conocimientos para inscribirse.
- Analista: consulta los datos y prepara los indicadores I1, I2 e I3.

La API implementa estas operaciones de negocio; no incluye autenticacion ni control de permisos por actor. Los actores describen responsabilidades del dominio.

Alcance

Base documental MongoDB gestion_eventos con colecciones eventos e inscripciones; modelo mixto con datos embebidos y referencias; FastAPI, PyMongo y Docker Compose con un replica set rs0 de un nodo. La demo HTTP usa Postman.

Se contemplan eventos gratuitos/pagos, invitaciones, actividades de distintos tipos, requisitos de conocimientos, cupos y cancelacion con historial. No se implementan cobros reales, devoluciones automaticas, notificaciones, autenticacion ni una coleccion independiente de usuarios.

Los datos son ficticios. El poblado reproducible tiene 20 eventos y 200 inscripciones. Al finalizar las pruebas manuales del 29/10/2026 hay 22 eventos y 201 inscripciones: se conservaron un evento de demo cancelado, un borrador extra y una inscripcion cancelada. Los nueve controles de consistencia dieron cero.

Criterio de exito

La API debe respetar reglas de negocio y atomicidad entre inscripcion/cupo y cancelacion/restitucion. Las consultas deben responder preguntas del dominio; los indices deben justificarse con planes y metricas; el analisis debe definir indicadores, denominadores y tratamiento de estados antes de graficar.

Requerimientos funcionales

- Crear un evento borrador; consultar, listar y filtrar eventos; publicar y actualizar campos permitidos.
- Eliminar únicamente borradores sin inscripciones y cancelar eventos conservando su historial.
- Inscribir usuarios por DNI cuando el evento esté publicado, sea futuro, tenga cupo y se cumplan todos los conocimientos requeridos.
- Cancelar inscripciones y restituir cupo una sola vez.
- Consultar demanda por actividad, ocupación y recaudación bruta registrada.

La actualización implementada modifica título, categoría, precio, capacidad, estado de publicación y conocimientos requeridos. No ofrece edición arbitraria de sede, fechas o cronograma en PATCH. Los actores son responsabilidades del dominio; no se implementó autenticación por roles.

1 Diseño documental y diccionario

Colecciones y relaciones

Coleccion	Unidad de documento	Embebidos	Referencias
eventos	Un evento	sede y actividades[] con detalles por tipo	Ninguna obligatoria
inscripciones	Una inscripcion de un usuario a un evento	usuario y evento_resumen	evento_id apunta a eventos._id

La sede y el cronograma se consultan junto al evento y tienen tamano acotado: embebidos permiten recuperar su detalle sin joins. Las inscripciones se separan porque su cantidad puede crecer y necesitan consulta y modificacion independientes. No se coloca un array creciente de personas dentro de cada evento.

usuario contiene los datos aportados para esa inscripcion. No es una referencia a una coleccion de usuarios ni existe id_usuario. El DNI, como string para no perder ceros iniciales, identifica al usuario en el dominio. _id identifica la INSCRIPCION. El par (evento_id, usuario.dni) tiene unicidad global en la coleccion, incluyendo inscripciones canceladas: cancelar no permite reinscribir el mismo DNI en el mismo evento con la regla actual.

Un mismo DNI puede figurar en distintos eventos. Sus datos son copias embebidas; no existe sincronizacion automatica de perfiles entre inscripciones. evento_resumen es una copia historica del titulo y fecha al inscribirse; evento_id permite consultar la version actual del evento. Editar un evento no reescribe las copias historicas.

Ejemplos en Extended JSON

Ejemplos ilustrativos del modelo, no nuevas inserciones ni exportaciones de la base:

```
{
 "_id": {"$oid": "6715a1b2c3d4e5f6a7b8c9d0"},
 "titulo": "Taller de MongoDB",
 "categoria": "Tecnología",
 "estado": "publicado",
 "fecha_inicio": {"$date": "2026-11-15T12:00:00Z"},
 "fecha_fin": {"$date": "2026-11-15T16:00:00Z"},
 "precio_entrada": {"$numberDecimal": "15000.00"},
 "capacidad_total": 30,
 "vacantes_disponibles": 29,
 "conocimientos_requeridos": ["mongodb", "docker"],
 "sede": {
  "nombre": "Centro de Capacitación",
  "direccion": "Av. Colón 100",
  "ciudad": "Córdoba",
  "capacidad_maxima": 100
 },
 "actividades": [{
  "codigo_actividad": "ACT-01",
  "titulo": "Introducción al modelado documental",
  "tipo": "taller",
  "orador": "Ana Pérez",
  "inicio": {"$date": "2026-11-15T12:00:00Z"},
  "fin": {"$date": "2026-11-15T14:00:00Z"},
  "sala": "Aula 1",
  "detalles_especificos": {
   "requiere_notebook": true,
   "software_previo": ["MongoDB Compass"],
   "nivel": "inicial"
  }
```

```
  }],
  "etiquetas": ["mongodb", "bases-de-datos"]
}
{
  "_id": {"$oid": "730000000000000000000000"},
  "evento_id": {"$oid": "6715a1b2c3d4e5f6a7b8c9d0"},
  "evento_resumen": {
    "titulo": "Taller de MongoDB",
    "fecha_inicio": {"$date": "2026-11-15T12:00:00Z"}
  },
  "usuario": {
    "dni": "44123456",
    "nombre_completo": "Juan Pérez",
    "email": "juan@example.com",
    "conocimientos": ["mongodb", "docker"]
  },
  "fecha_inscripcion": {"$date": "2026-10-11T12:00:00Z"},
  "estado_inscripcion": "confirmada",
  "tipo_entrada": "general",
  "importe_abonado": {"$numberDecimal": "15000.00"},
  "metodo_pago": "transferencia",
  "actividades_seleccionadas": ["ACT-01"]
}
```


Diccionario: eventos

Campo	Tipo BSON / estructura	Regla
_id	ObjectId	Identificador del evento
titulo, categoria	String	Textos no vacios
estado	String	borrador, publicado, finalizado o cancelado; API crea borrador
fecha_inicio, fecha_fin	Date	Fin posterior al inicio; entradas API con zona horaria
precio_entrada	Decimal128	No negativo; entrada API con hasta dos decimales
capacidad_total	Entero	Positivo y no superior a sede.capacidad_maxima
vacantes_disponibles	Entero	capacidad_total menos confirmadas; no negativo
conocimientos_requeridos	Array de strings	Siempre presente; puede estar vacio; minusculas y sin espacios al borde
sede	Documento	nombre, direccion, ciudad y capacidad_maxima; servicios_disponibles opcional en poblado
actividades	Array de documentos	Codigos unicos dentro del evento y horarios dentro de sus limites
actividades[].tipo	String	taller, conferencia o panel; detalles_especificos depende del tipo
etiquetas	Array de strings	Opcional en documentos de carga

Campo	Tipo BSON / estructura	Regla
requisitos_generales	String	Opcional; texto descriptivo, diferente del array de conocimientos

Diccionario: inscripciones

Campo	Tipo BSON / estructura	Regla
_id	ObjectId	Identifica la inscripcion
evento_id	ObjectId	Referencia a un evento existente
evento_resumen	Documento	titulo y fecha_inicio como copia historica
usuario	Documento	Reemplaza al campo anterior participante
usuario.dni	String	Digitos; identificador del usuario; unicidad por evento
usuario.nombre_completo	String	No vacio
usuario.email	String	Validado como email en la API
usuario.institucion	String	Opcional
usuario.conocimientos	Array de strings	Minusculas, recortados y sin duplicados; puede estar vacio
fecha_inscripcion	Date	La genera la API al confirmar
estado_inscripcion	String	confirmada o cancelada
tipo_entrada	String	general o invitacion
importe_abonado	Decimal128	Lo calcula el servidor; cero si es invitacion
metodo_pago	String	efectivo o transferencia; obligatorio si importe > 0
actividades_seleccionadas	Array de strings	Codigos del evento; sin repetir; puede estar vacio
preferencias_alimentarias	Array de strings	Opcional
fecha_cancelacion	Date	Se agrega al cancelar

Regla de conocimientos

Para confirmar una inscripcion debe cumplirse:

$$\text{conocimientos_requeridos del evento} \subseteq \text{usuario.conocimientos}$$

Se exigen TODOS los requisitos. Un evento con [] acepta cualquier conjunto de conocimientos. La API normaliza strings, rechaza vacios y quita repetidos. Si faltan conocimientos devuelve 409 con detail.conocimientos_faltantes. El catalogo fijo de ocho valores se usa en el generador; la API no limita los conocimientos a ese catalogo. Editar requisitos se rechaza si invalida alguna inscripcion confirmada existente.

Atomicidad y consistencia

Al inscribirse se descuenta cupo con filtro por publicado, fecha futura y vacantes positivas; se recupera el evento y, usando la misma sesion, se validan requisitos, actividades, duplicado y pago, y se inserta la inscripcion. Una excepcion aborta ambas escrituras. El indice unico es la garantia de unicidad concurrente.

Cancelar marca la inscripcion y restituye una vacante en una transaccion. Repetir la cancelacion es idempotente: devuelve 200 sin sumar otra vacante. Se usa snapshot, write concern majority y lectura del primario. Los callbacks no tienen efectos externos para permitir reintentos del driver.

La recarga es diferente: el drop y la preparacion de colecciones/indice quedan FUERA de la transaccion; toda la insercion queda dentro. Un fallo de insercion no recupera el contenido borrado por el drop. Se requiere backup antes de recargar.

2 Consultas MongoDB

En scripts/03_consultas.js cada consulta imprime enunciado, requisito cubierto y resultado esperado. Todas son de solo lectura. La evidencia real esta en anexo/consultas_modelo.txt (regenerada tras corregir la agrupacion de la consulta 7).

N.°	Pregunta	Herramienta principal	Resultado inicial verificado
1	Eventos con precio entre 4000 y 8000	$gte, $lte, proyeccion, sort	9 eventos
2	Eventos de noviembre de 2026	Rango ISODate	15 eventos
3	Publicados con vacantes, gratuitos o de Tecnologia	AND implicito, $or	4 eventos
4	Eventos en Cordoba	Notacion sede.ciudad	5 eventos
5	Eventos con taller intermedio	$elemMatch	10 eventos
6	Proximos cinco publicados	Fecha actual, sort, limit	5 en la fecha de prueba; depende del dia
7	Confirmadas que seleccionan cada actividad de cada evento	$match, $unwind, $group	35 pares evento/actividad
8	Recaudacion bruta mensual por fecha de inscripcion	$dateTrunc, $sum	Agosto: 234000; septiembre: 0; octubre: 555000
9	Promedio, minimo y maximo de precios por categoria	$avg, $min, $max	4 categorias, 5 eventos por categoria
10	Confirmadas y ocupacion por evento	$lookup y porcentaje	20 eventos; Emprendimientos 3 al 100%
11	Publicados compatibles con javascript y git	$expr, $setIsSubset	5 eventos, incluidos requisitos vacios
12	Conocimientos mas frecuentes en confirmadas	$unwind, $group	8 conocimientos; excel primero con 72

Los valores de esta tabla corresponden al poblado inicial de 20/200, no al estado posterior de 22/201. La demo aporta eventos y una inscripcion cancelada y puede alterar conteos de eventos/precios. Los indicadores finales se

calcularon despues de la demo y se presentan en la seccion 5. La consulta 10 incluye todos los estados como control; I2 del analisis excluye borradores y cancelados.

ACT-01/ACT-02 son codigos locales a cada evento, no identificadores globales de actividades. Por eso I1 agrupa por evento_id y codigo, no solamente por codigo. Las frecuencias de conocimientos cuentan inscripciones, no personas distintas entre eventos, y cada inscripcion puede aportar a varios conocimientos.

Los comandos completos, comentarios y resultados esperados están en scripts/03_consultas.js; su salida real se conserva en docs/anexo/consultas_modelo.txt. El archivo de consultas no crea índices ni modifica documentos.

3 API y consistencia

Arquitectura

main.py construye la aplicacion; routes/http.py maneja HTTP y serializacion; schemas/modelos.py valida entradas; services aplica reglas y orquesta transacciones; repositories es el unico lugar que consulta o escribe colecciones. db.py gestiona conexion y sesiones. Uvicorn sirve FastAPI; /docs es Swagger UI generado por FastAPI.

Dockerfile usa Python 3.12, instala api/requirements.txt y ejecuta Uvicorn sin privilegios. Compose conecta la API a mongo:27017 y espera un primario saludable. El volumen externo existente conserva los datos al recrear contenedores.

Endpoints

Metodo	Ruta	Exito
GET	/estado	200
GET	/eventos	200
GET	/eventos/{id}	200
POST	/eventos	201
PATCH	/eventos/{id}	200
DELETE	/eventos/{id}	204, sin cuerpo
PATCH	/eventos/{id}/cancelacion	200
POST	/eventos/{id}/inscripciones	201
GET	/eventos/{id}/inscripciones	200
PATCH	/inscripciones/{id}/cancelacion	200

GET /eventos admite categoria, ciudad, estado, fecha_desde, fecha_hasta, precio_minimo, precio_maximo, conocimiento, etiqueta, con_vacantes y limite. conocimiento busca eventos que EXIGEN ese conocimiento puntual; no comprueba por si solo si una persona cumple TODOS los requisitos.

Errores: 404 inexistente; 409 conflicto de negocio; 422 entrada o ID invalido; 503 fallo de acceso a base o indice del nuevo modelo ausente. Las fechas se reciben con zona horaria. La API calcula importe y cupos; no confia esos valores al cliente.

Pruebas manuales del 29/10/2026

La coleccion postman/gestion_eventos.postman_collection.json tiene 24 peticiones ordenadas. Guarda IDs y construye fechas futuras. La ejecucion fue realizada manualmente por el usuario, quien informo coincidencia con los resultados esperados. Se observaron capturas de 404 y 422; esos errores eran casos intencionales. No se dispone de una exportacion consolidada de los resultados de tests de Postman.

Casos principales: 409 por conocimientos faltantes con cupo conservado; 201 por inscripcion valida y descuento; 409 por cupo agotado; cancelacion y repeticion sin sumar dos veces; 409 por DNI duplicado con cupo disponible; borrado de borrador, 404 posterior y 422 por rango invalido. La verificacion final en MongoDB dio cero en los nueve controles, con 22 eventos y 201 inscripciones.

La demo conservo un evento cancelado y su inscripcion cancelada, mas un borrador extra. No se identifica la causa exacta del borrador extra con la evidencia actual; se documento en lugar de borrarlo. No hay inscripciones asociadas a borradores.

Esto valida el comportamiento secuencial observado. No se realizaron pruebas de carga concurrente, caida de servidor ni eleccion de un nuevo primario. El replica set de un nodo habilita transacciones pero no aporta redundancia.

Preparar la defensa

Mostrar /estado, una consulta filtrada, el 409 por conocimientos faltantes y las consultas de cupo antes/despues; contrastar con la inscripcion valida. Mostrar la evidencia de indices y los nueve controles finales. Un 404/409/422 esperado es un resultado correcto, aunque Postman lo coloree en rojo.

Caso transaccional de inscripción

La reserva usa find_one_and_update con filtros de estado publicado, fecha futura y vacantes mayores que cero, junto a $inc -1. El servicio valida sobre el documento recuperado actividades, conocimientos, pago y duplicado, e inserta con la misma sesión. Ante una excepción, la transacción aborta y revierte el descuento. DuplicateKeyError se captura fuera del callback para no ocultar el aborto.

La cancelación actualiza el estado y restituye una vacante en una transacción. Si ya estaba cancelada, devuelve el mismo estado sin incrementar nuevamente. Se usa read concern snapshot y write concern majority. Los callbacks evitan efectos externos porque el driver puede reintentarlos.

4 Índices y planes de ejecución

Mediciones observadas el 29/10/2026 despues de cargar 20 eventos y 200 inscripciones, antes de agregar la demo de Postman. Salida completa en anexo/indices_modelo.txt.

Cada prueba conserva filtro, orden y proyeccion. hint({$natural:1}) fuerza el recorrido natural, sin borrar indices; hint(nombre) fuerza el indice candidato. La comparacion no prueba que el optimizador lo elija sin hint ni garantiza menor tiempo con un dataset tan pequeno. No se midieron tiempos para inferir aceleraciones.

Resultados observados

Indice	Plan sin / con indice	Docs sin / con	Claves sin / con	Devueltos en ambos
estado_ciudad_fecha	SORT + COLLSCAN / FETCH + IXSCAN	20 / 4	0 / 4	4
evento_usuario_dni_unico	COLLSCAN / FETCH + IXSCAN	200 / 1	0 / 1	1
conocimientos_requeridos_multikey	COLLSCAN / FETCH + IXSCAN	20 / 4	0 / 4	4

1. Estado, ciudad y fecha

```
{ estado: 1, "sede.ciudad": 1, fecha_inicio: 1 }
```

Consulta: publicados en Cordoba con fecha_inicio en noviembre de 2026; sort por fecha_inicio ascendente. La igualdad en los dos primeros campos permite aprovechar el rango y orden del tercero. El plan indexado revisa cuatro documentos y evita la etapa SORT que aparecia en el recorrido natural. isMultiKey: false.

Costo: espacio y mantenimiento por inserciones o cambios de estado, sede.ciudad y fecha_inicio. No optimiza por igual todas las combinaciones de filtros de la API.

2. Unicidad por evento y DNI

```
{ evento_id: 1, "usuario.dni": 1 }
// unique: true, name: "evento_usuario_dni_unico"
```

Consulta exacta por evento_id del Taller de MongoDB y usuario.dni 44123456. El plan indexado examina una clave y un documento frente a 200 documentos del recorrido natural. isUnique: true, isMultiKey: false.

Justificacion: busqueda frecuente e integridad; impide duplicar un DNI en un evento, incluso si la inscripcion anterior esta cancelada. Reemplaza al indice sobre participante.dni. Costo: almacenamiento, mantenimiento y comprobacion de unicidad.

Se intento insertar una copia con OTRO _id y el mismo evento/DNI. MongoDB rechazo la operacion con 11000 por el indice compuesto. La prueba se ejecuto en una transaccion y los totales antes/despues fueron 200/200.

3. Requisitos de conocimientos (multikey)

```
{ conocimientos_requeridos: 1 }
// name: "conocimientos_requeridos_multikey"
```

Consulta: eventos que requieren javascript. isMultiKey: true y multiKeyPaths identifica conocimientos_requeridos como array. El plan examina cuatro claves y cuatro documentos, con los mismos cuatro resultados del recorrido de 20 documentos.

Costo: genera claves por elementos del array y debe mantenerlas al modificarlo. Esta medicion corresponde a igualdad por un conocimiento; NO demuestra que el indice optimice el $setIsSubset usado para elegibilidad.

Lectura de metricas

- totalDocsExamined: documentos que el plan debio examinar.
- totalKeysExamined: entradas del indice examinadas; 0 en recorrido natural.
- nReturned: resultados entregados; debe coincidir entre los dos planes.
- FETCH: recupera documentos a partir de claves; estas no son consultas cubiertas.

Los indices automaticos de _id tambien existen; los tres anteriores son los indices adicionales del proyecto.

5 Indicadores y visualizaciones

Se usó la base posterior a la demo, de 22 eventos y 201 inscripciones. Los indicadores se ejecutaron en instantáneas de lectura independientes. Las fechas y totales quedaron registrados en corte.json; no se reportaron escrituras entre ejecuciones. Igualdad de conteos no prueba por sí sola identidad completa de datos.

Indicador	Corte UTC del 29 de octubre	Unidad
I1	21:27:27	Selecciones por evento y actividad
I2	21:33:39	Porcentaje de capacidad
I3	21:42:34	Importe registrado por mes

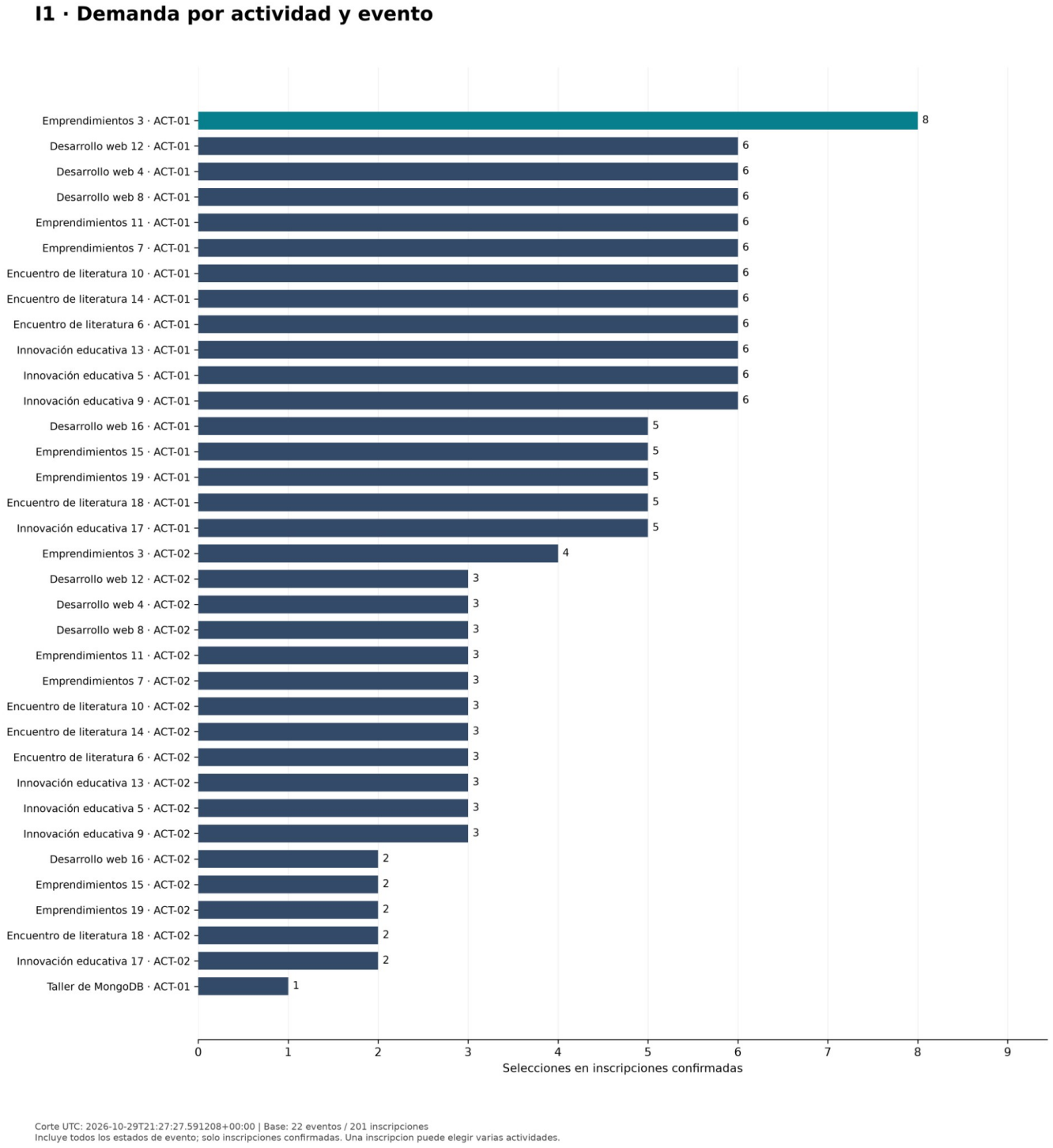
I1 Demanda por actividad

Se contaron 152 inscripciones confirmadas; 52 no seleccionaron actividades. Las otras 100 aportaron 147 selecciones en 35 pares de evento y actividad. Emprendimientos 3 / ACT-01 registra el máximo de 8. Una inscripción puede elegir varias actividades, por lo que selecciones y personas no son unidades equivalentes.

Se agrupa por evento_id y código de actividad; ACT-01 no identifica una actividad global. El indicador conserva demanda histórica de eventos cancelados si sus inscripciones siguen confirmadas. Los pares sin selecciones no aparecen en el gráfico.

Figura 1 Demanda por actividad y evento

Salida real generada por I1. Los 35 grupos con demanda positiva se muestran completos.



Corte UTC: 2026-10-29T21:27:27.591208+00:00 | Base: 22 eventos / 201 inscripciones
Incluye todos los estados de evento; solo inscripciones confirmadas. Una inscripcion puede elegir varias actividades.

I2 Ocupación por evento

Se incluyeron 17 eventos publicados o finalizados y se excluyeron 5 borradores/cancelados. La población elegible tiene 143 confirmadas, capacidad total de 682 y 539 vacantes.

Medida	Fórmula o base	Resultado
Ocupación global	100 × 143 / 682	20,97%
Promedio simple	Media de porcentajes por evento	25,12%
Eventos incluidos	Publicados y finalizados	17
Eventos excluidos	Borradores y cancelados	5

El porcentaje de cada evento se calcula como 100 × confirmadas / capacidad_total. La ocupación global pondera por capacidad; el promedio simple asigna el mismo peso a cada evento y por eso puede diferir. Los cálculos se realizan antes de redondear las filas.

Las nueve confirmadas del evento cancelado original explican la diferencia frente a las 152 consideradas por I1. En eventos finalizados, ocupación describe el registro final de reservas, no asistencia efectiva ni disponibilidad actual.

La tabla completa se conserva en analisis/resultados/I2_20261029T213339_071414Z/i2_ocupacion.csv. La interpretación y el corte acompañan a ese archivo.

I3 Recaudación bruta registrada por mes

Mes UTC	Importe bruto	Inscripciones	Canceladas
2026-08	234.000,00	55	15
2026-09	0,00	1	0
2026-10	555.000,00	145	34
Total	789.000,00	201	49

Se suman importes de todos los estados por mes UTC de fecha_inscripcion. Las 49 canceladas registran 144.000, incluidos sin deducir dentro del bruto. No se registran devoluciones ni se descuentan comisiones, impuestos o costos; el indicador no representa ganancia ni ingreso neto.

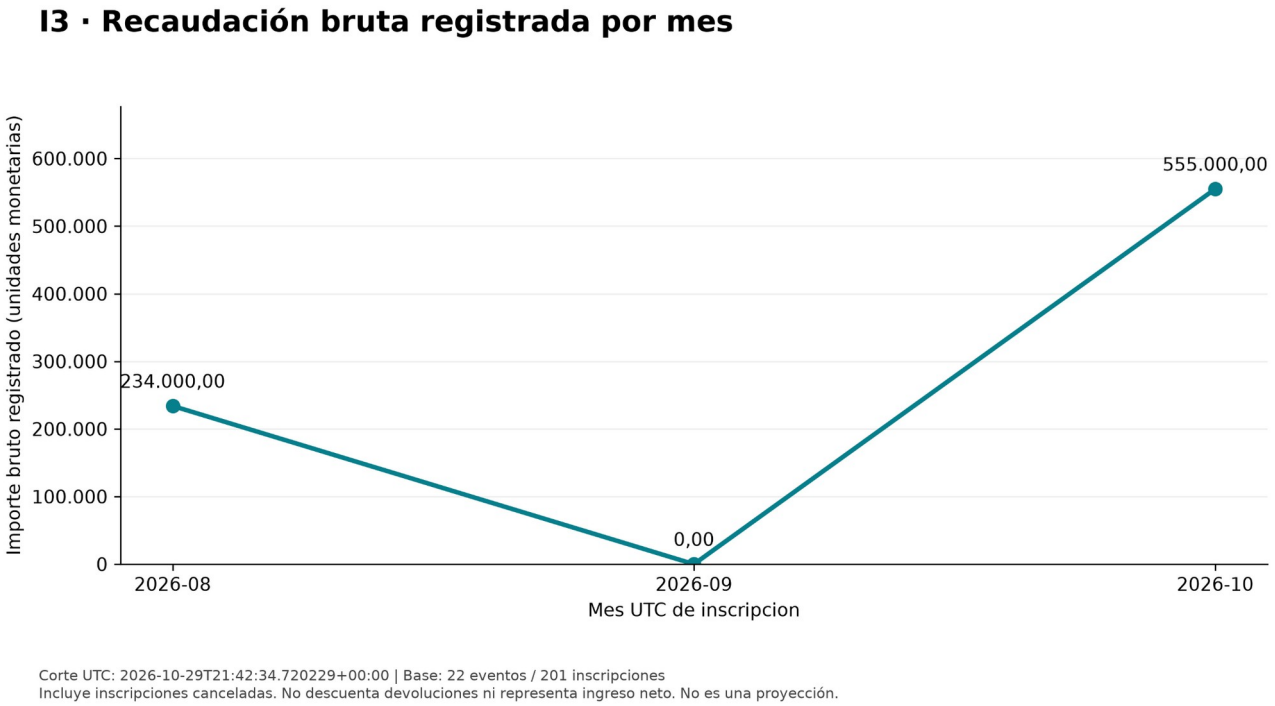


Figura 2. Totales mensuales observados. Septiembre contiene una inscripción de importe cero: cero recaudado no implica cero inscripciones. La línea no estima importes diarios ni proyecta períodos futuros.

6 Reproducción de la solución

El entorno se ejecutó desde la carpeta reorganizada, usando el proyecto Compose proyecto-eventos y el volumen persistente existente proyecto-eventos_mongo_datos. Se hizo down/up sin -v, se construyó la API y se comprobó /estado. Esto verifica continuidad sobre el volumen existente, no un arranque virgen sin volumen.

La API usa mongo:27017 dentro de Docker; el host y el notebook usan localhost:27018. El replica set rs0 de un nodo permite transacciones, pero no redundancia. La configuración usa directConnection=true para este entorno local de un nodo.

```
docker compose -p proyecto-eventos up -d --build
```

```
docker compose -p proyecto-eventos exec -T mongo mongosh --quiet --file /scripts/04_verificaciones.js
```

```
python -m jupyter nbconvert --to notebook --execute --inplace analisis/analisis.ipynb
```

El README documenta requisitos, provisionamiento de un volumen nuevo, arranque, respaldo, recarga, scripts, API, Postman y análisis. Los comandos de provisionamiento desde cero están documentados, pero no se ejecutaron en la PC verificada. No se afirma esa prueba como realizada.

Antes de la recarga se obtuvo un mongodump y se comprobó una copia externa de 112.073 bytes. No se ensayó su restauración. La recarga borra eventos e inscripciones antes de la transacción de inserción; un aborto posterior no recupera el contenido borrado.

El notebook quedó guardado con 6 celdas de código ejecutadas y 0 errores. Conserva tablas, figuras y las interpretaciones producidas. Los scripts permiten reproducir cada indicador por separado sin escribir en MongoDB.

7 Conclusiones y anexo de evidencias

El modelo documental responde a dos patrones: recuperar el evento completo con su sede y cronograma, y gestionar inscripciones independientes. El DNI identifica al usuario en el dominio, mientras el índice compuesto impide repetirlo en un mismo evento. La copia histórica de evento evita reescribir las condiciones originales de inscripción.

Las operaciones secuenciales de la demo y la auditoría final fueron consistentes. Los índices redujeron documentos examinados conservando resultados; no se midió una aceleración temporal. El análisis diferencia demanda, ocupación y registro monetario, evitando tratarlos como medidas intercambiables.

El poblado ficticio y pequeño permite demostrar el diseño y las técnicas; no justifica inferencias de mercado. No se probaron fallas de servidor, carga concurrente, restauración ni elecciones de primario. Esas pruebas, junto con autenticación y pagos reales, quedan fuera del alcance implementado.

Evidencias del repositorio

Evidencia	Contenido comprobado
docs/anexo/recarga_modelo.txt	Carga inicial 20 eventos y 200 inscripciones
docs/anexo/verificaciones_modelo.txt	Nueve controles iniciales en cero
docs/anexo/consultas_modelo.txt	Doce consultas y 35 grupos en consulta 7
docs/anexo/indices_modelo.txt	COLLSCAN/IXSCAN y duplicado 11000; total 200/200
docs/anexo/verificaciones_post_postman.txt	Estado final 22/201; nueve controles en cero
analisis/analisis.ipynb	Seis celdas ejecutadas y cero errores
analisis/resultados/	CSV, cortes, interpretaciones y figuras reales

Postman se ejecutó manualmente con 24 peticiones. Los resultados esperados fueron confirmados por el usuario y se observaron capturas de los casos 404 y 422. No se presenta un reporte consolidado del Runner que no fue exportado. La demo agregó un evento cancelado, una inscripción cancelada y un borrador extra, identificado y conservado sin inconsistencia.

Fuentes y trazabilidad

Consigna Proyecto NoSQL MongoDB, apartados 5 a 8; fuentes del proyecto en la rama cambio-modelo-y-estructura; resultados y capturas de las ejecuciones locales del 29/10/2026. Commits comprobados: 3cb55a7 para modelo/API y b881392 para análisis y notebook. La publicación remota de esos commits seguía pendiente al cierre.

Documentación técnica de referencia:
mongodb.com/docs/languages/python/pymongo-driver/current/crud/transactions/;
docs.docker.com/reference/compose-file/volumes/;
learning.postman.com/docs/tests-and-scripts/write-scripts/postman-sandbox-reference/pm-variables/.